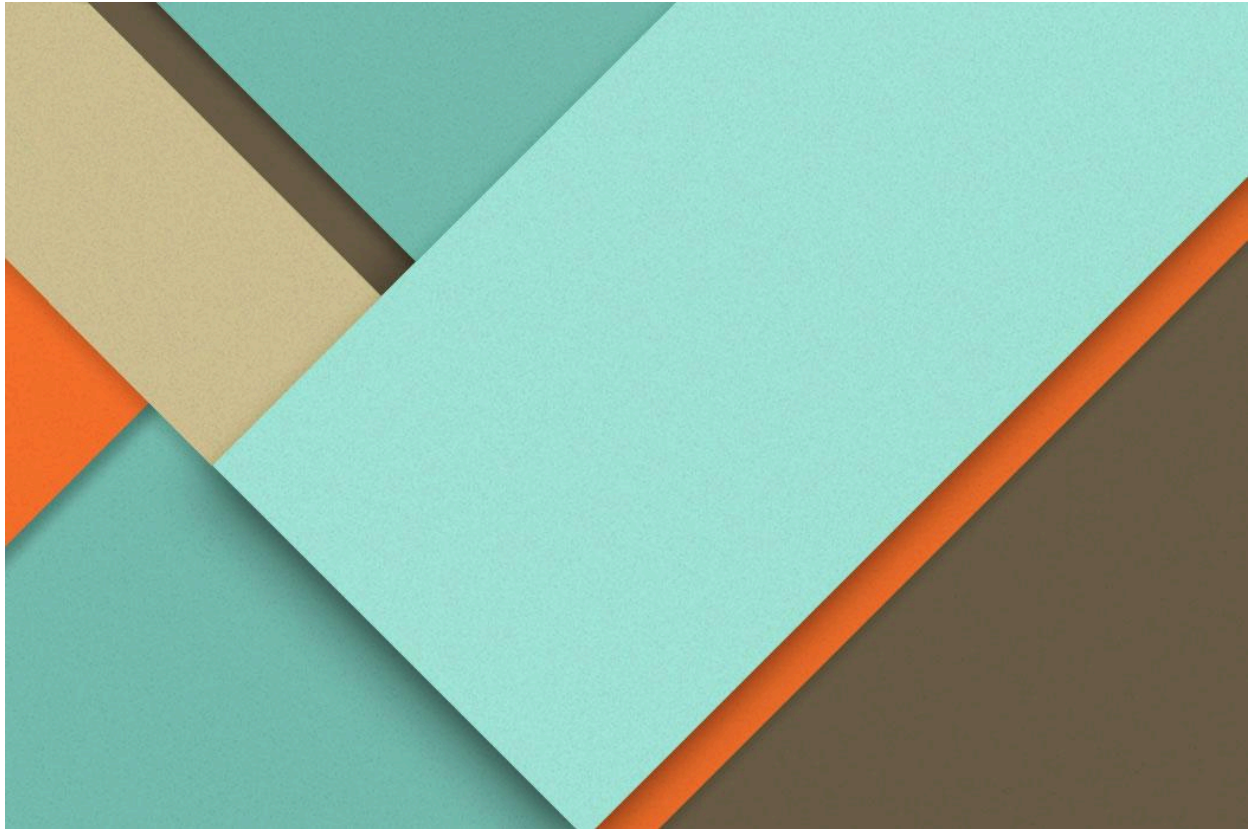




Manual YourHome Beta

20/01/2026

—

José Manuel Sánchez Rosal

2º DAM

IES Antonio Gala

1400 Palma del Río (Córdoba)

Índice

Índice.....	0
1. Descripción General.....	1
2. Esquema de Navegación.....	1
3. Arquitectura de Datos y Room (El intermediario).....	2
3.1 La Entidad (Casa.kt).....	2
3.2 El DAO (CasaDao.kt).....	2
3.3 El ViewModel (CasaViewModel.kt).....	3
4. Mejoras y extras Implementados.....	3

1. Descripción General

"Casas App" es una aplicación móvil nativa desarrollada en Kotlin y Android Studio que permite la gestión integral de un catálogo de viviendas. La aplicación sigue el patrón de arquitectura **MVVM (Model-View-ViewModel)** para asegurar una separación limpia entre la lógica de negocio, la persistencia de datos y la interfaz de usuario.

2. Esquema de Navegación

La navegación de la aplicación se gestiona mediante el componente **NavHost** de Jetpack Compose, que actúa como un contenedor que intercambia los diferentes "destinos" (pantallas) según las acciones del usuario.

El flujo de navegación se estructura de la siguiente manera:

- **Pantalla de Inicio (**startDestination**)**: Es el punto de entrada. Desde aquí, el usuario tiene dos vías principales:
 - **Ver Catálogo**: Navega hacia la **PantallaGaleria**.
 - **Añadir Casa**: Navega hacia la **PantallaFormulario** con un ID por defecto (0) para indicar que es un registro nuevo.

- **Pantalla Galería:** Muestra un listado de viviendas (Cards) con su foto principal, precio y dirección.
 - **Clic en la tarjeta:** Navega a `PantallaDetalle` pasando el `id` de la casa como argumento (`navArgument`).
 - **Clic en editar:** Navega a `PantallaFormulario` pasando el `id` de la casa existente para cargar sus datos.
- **Pantalla Formulario:** Reutilizable para creación y edición.
 - Al guardar los datos, utiliza `popBackStack()` para devolver al usuario a la pantalla anterior, actualizando la lista automáticamente.
- **Pantalla Detalle:** Recibe el `id`, solicita la información a la base de datos y muestra toda la información extendida y el carrusel de imágenes.

3. Arquitectura de Datos y Room (El intermediario)

Para garantizar que los datos sobrevivan al cierre de la aplicación, se utiliza la librería **Room**, que actúa como una capa de abstracción sobre la base de datos nativa SQLite de Android.

El funcionamiento del sistema de persistencia se basa en tres componentes clave que trabajan en conjunto:

3.1 La Entidad (Casa.kt)

Define la estructura de la tabla en la base de datos. Cada instancia de la clase `Casa` representa una fila en la tabla `casas`.

- **Campos:** Incluye datos primitivos (`id`, `nombre`, `precio`, `esAlquiler`) y una lista de rutas de imágenes (`List<String>`).
- **TypeConverters:** Dado que SQLite no permite guardar listas directamente, se ha implementado un conversor que transforma la lista de imágenes en una cadena de texto plana para guardarla, y la reconvierte a lista al leerla.

3.2 El DAO (CasaDao.kt)

Es la interfaz de acceso a datos (Data Access Object). Define los métodos para interactuar con la base de datos sin escribir SQL complejo:

- `insertar()`: Gestiona tanto la creación como la actualización de registros (`OnConflictStrategy.REPLACE`).
- `obtenerTodas()`: Retorna un flujo reactivo (`Flow<List<Casa>>`), lo que permite que la interfaz se actualice automáticamente en tiempo real si hay cambios en la base de datos.

3.3 El ViewModel (CasaViewModel.kt)

Actúa como intermediario entre la Base de Datos y la Interfaz (UI).

- Utiliza **Corrutinas de Kotlin** (`viewModelScope.launch`) para realizar operaciones de base de datos en un hilo secundario (IO), evitando que la aplicación se congele.
- **Gestión de Imágenes:** Implementa una lógica personalizada para copiar las imágenes desde la galería temporal del dispositivo al almacenamiento interno de la app, asegurando que las fotos permanezcan accesibles permanentemente.

4. Mejoras y extras Implementados

Además de los requisitos base del enunciado, la aplicación incluye las siguientes características avanzadas:

1. **Edición de Registros:** Se ha implementado la capacidad de modificar viviendas existentes, reutilizando el formulario de creación.
2. **Gestión de Múltiples Imágenes:** Se permite seleccionar y visualizar varias fotos por vivienda, en lugar de una sola.
3. **Copia de Seguridad de Imágenes:** Solución al problema de persistencia de URIs mediante copia física de archivos al almacenamiento interno (`filesDir`).
4. **Campos Avanzados:** Inclusión de precio, selector de tipo (Alquiler/Venta), dirección y gestión dinámica de características (extras).